US 20250284566A1

(19) **United States**

(12) **Patent Application Publication**

Dubey et al.

(10) **Pub. No.: US 2025/0284566 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **TENANCY CONTROL PLANE FOR SAAS APPLICATIONS**

(71) Applicant: **CIENA CORPORATION**, Hanover, MD (US)

(72) Inventors: **Pradeep Dubey**, Haryana (IN); **Rajesh Kalra**, Haryana (IN); **Sandeep Kumar**, New Delhi-87 (IN); **Neeraj Shrivastava**, Haryana (IN)

(73) Assignee: **CIENA CORPORATION**, Hanover, MD (US)

(21) Appl. No.: **18/643,351**

(22) Filed: **Apr. 23, 2024**

(30) **Foreign Application Priority Data**

Mar. 9, 2024 (IN) 202411017023

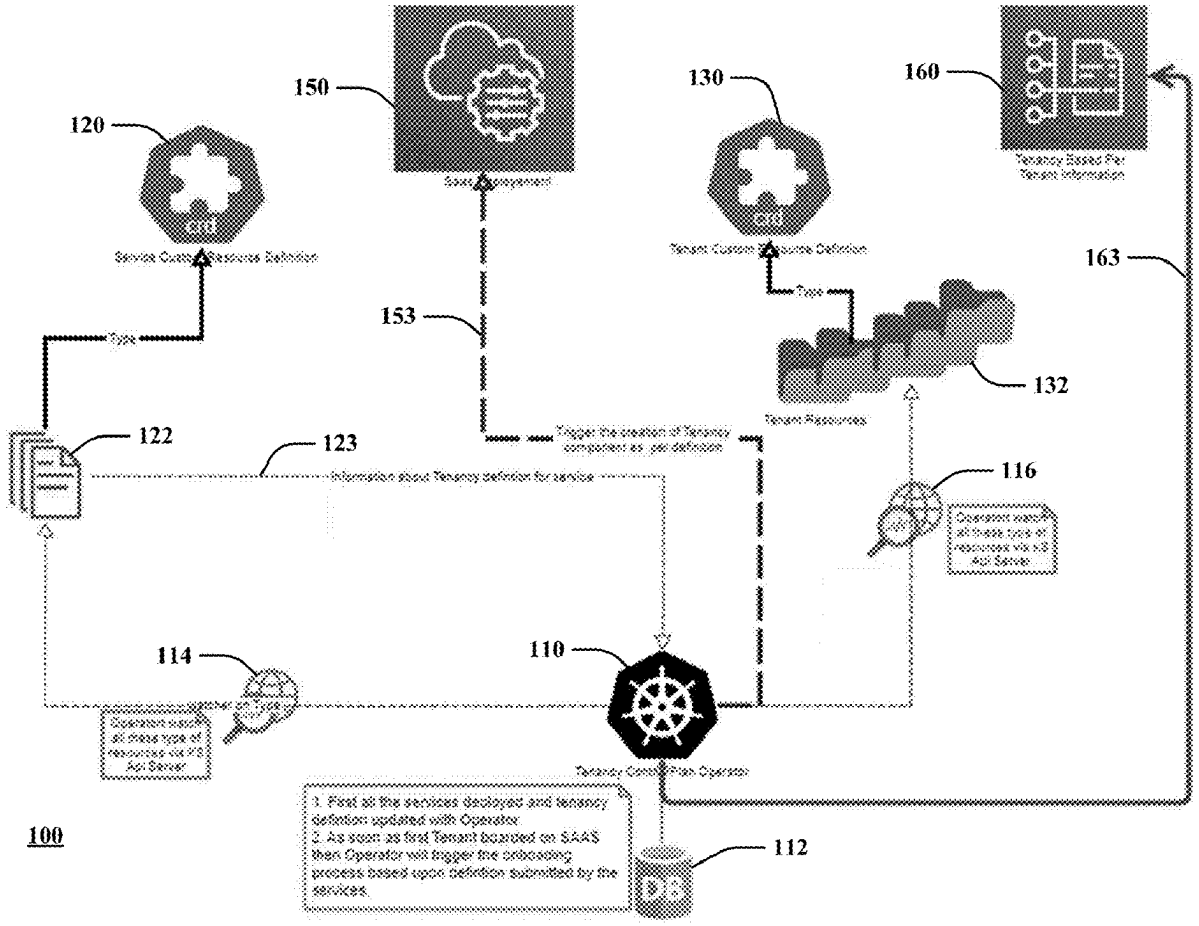
Publication Classification

(51) **Int. Cl.**
G06F 9/54 (2006.01)

(52) **U.S. Cl.**
CPC ***G06F 9/542*** (2013.01)

(57) **ABSTRACT**

Aspects of the subject disclosure may include, for example, the deployment of containerized multi-tenancy Software-as-a-Service (SaaS) applications with multiple services. Each of the multiple services may define its own multi-tenancy criteria through the creation of service custom resource definitions. A tenant control plane operator may fetch and persist the service custom resource definitions created by the services. The SaaS application may create tenant custom resource definitions as part of onboarding a new tenant. The tenant control plane operator may then alert each of the services that a tenant is being onboarded so that each service may create tenant specific resources to support multi-tenancy. Other embodiments are disclosed.



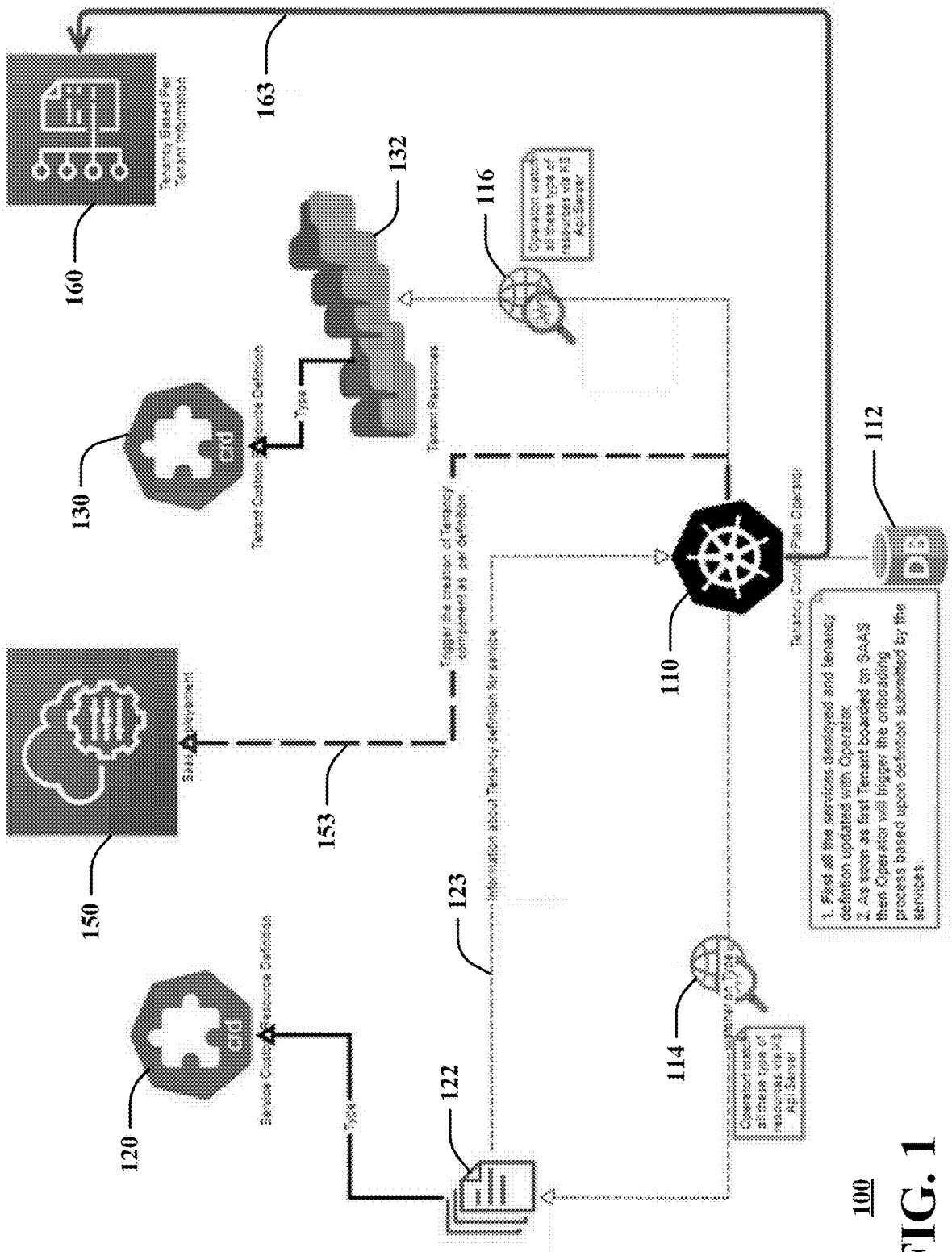


FIG. 1

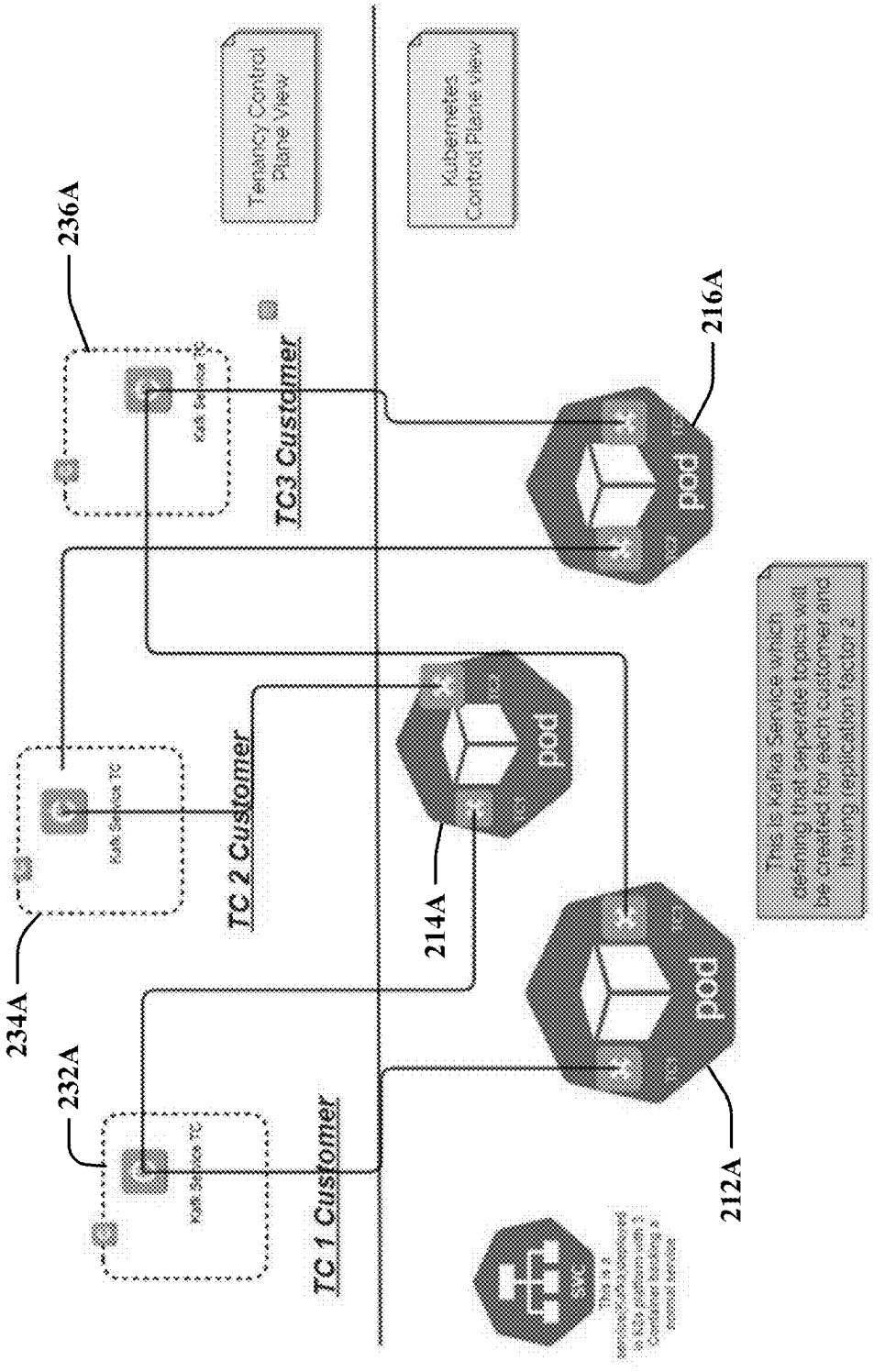

Tenancy Control Plane View
Kubernetes Control Plane View
236A
TC3 Customer
216A
pod
TC 2 Customer
214A
234A
232A
TC 1 Customer
212A

FIG. 2A

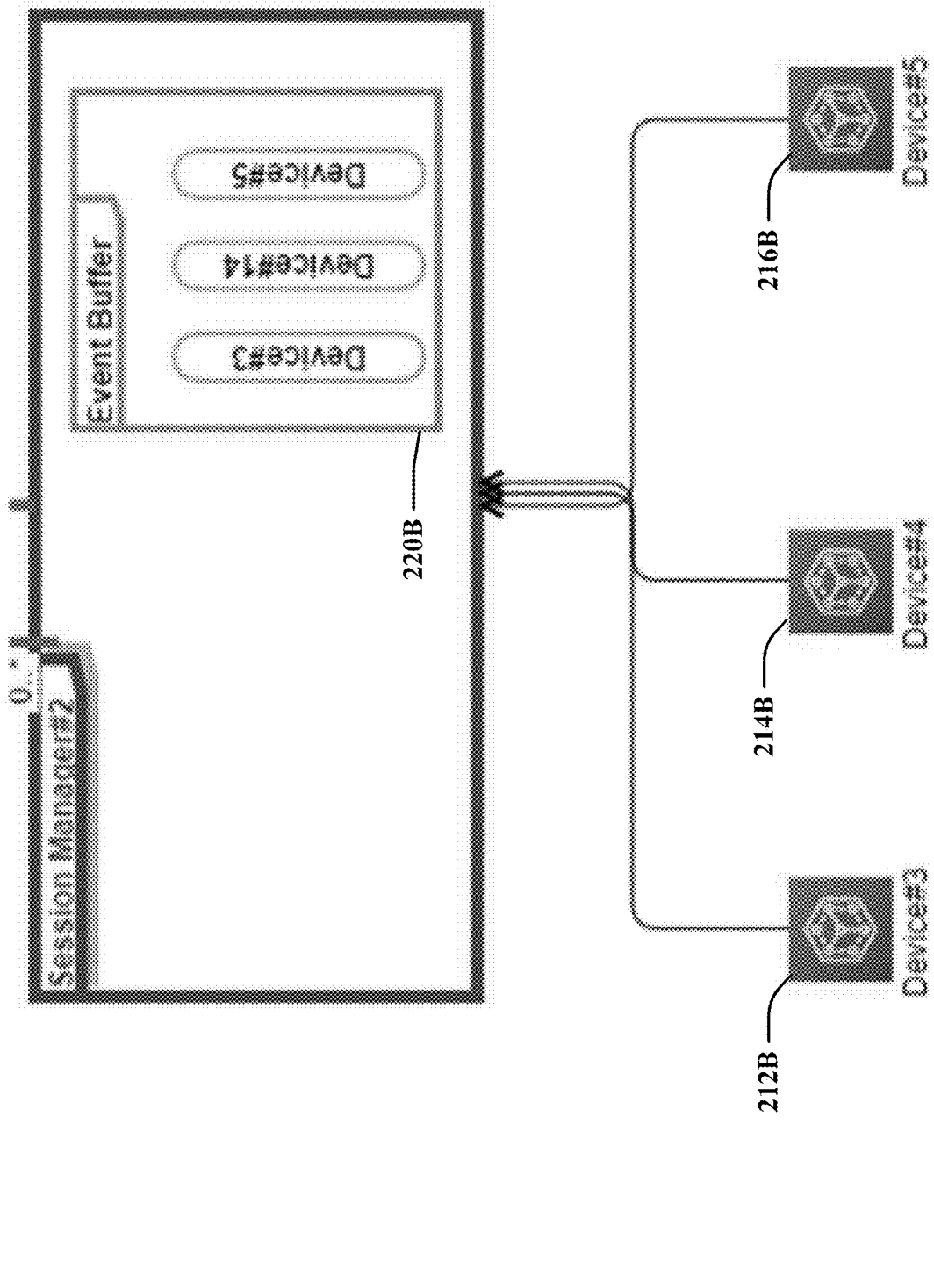

Session Manager#2
Event Buffer
Device#5
Device#14
Device#3
220B
216B
214B
212B
Device#5
Device#4
Device#3

FIG. 2B

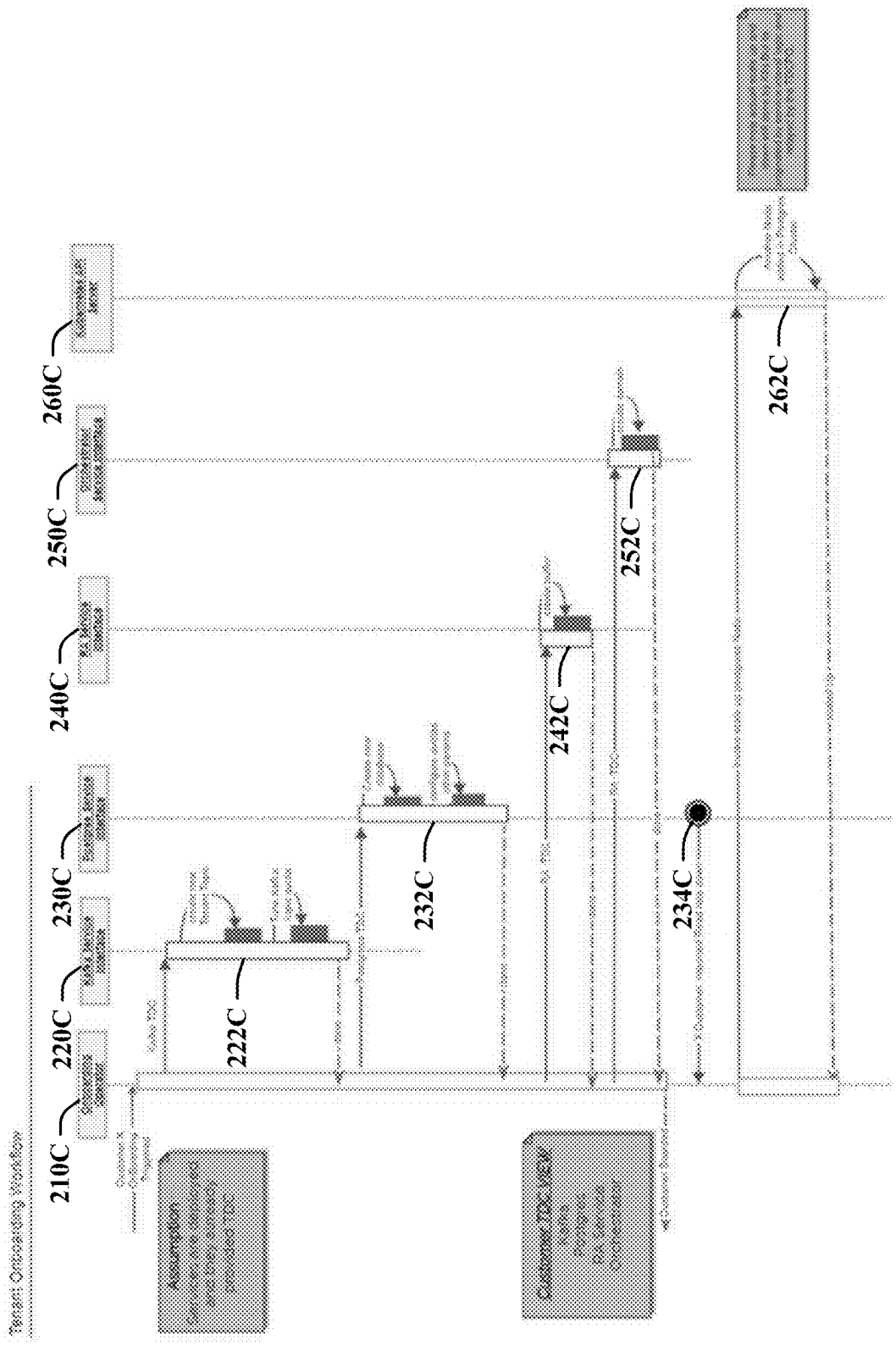
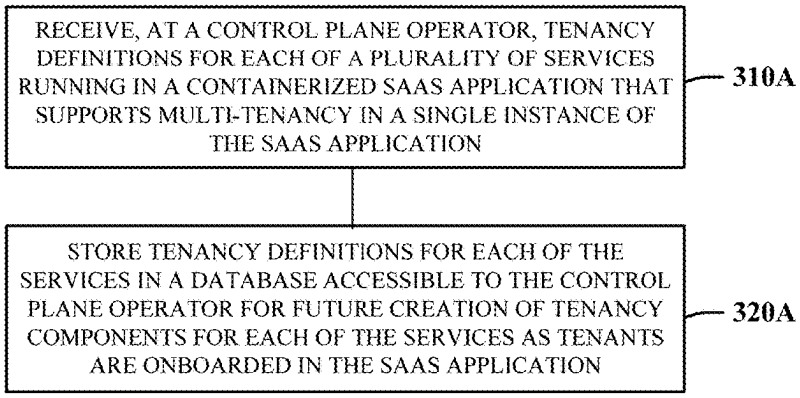
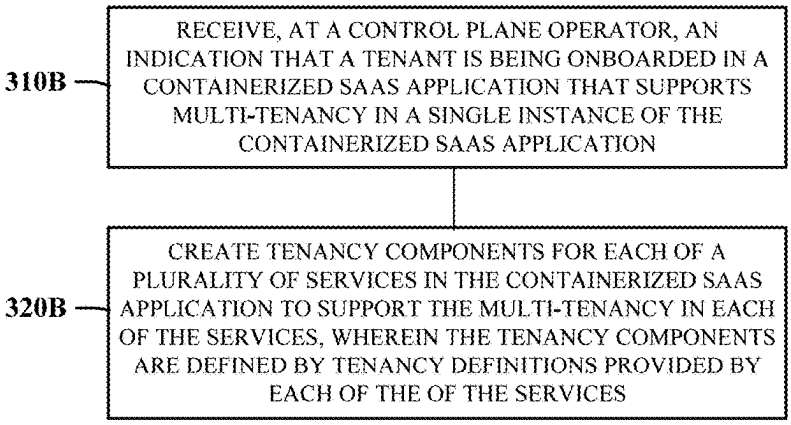


FIG. 2C



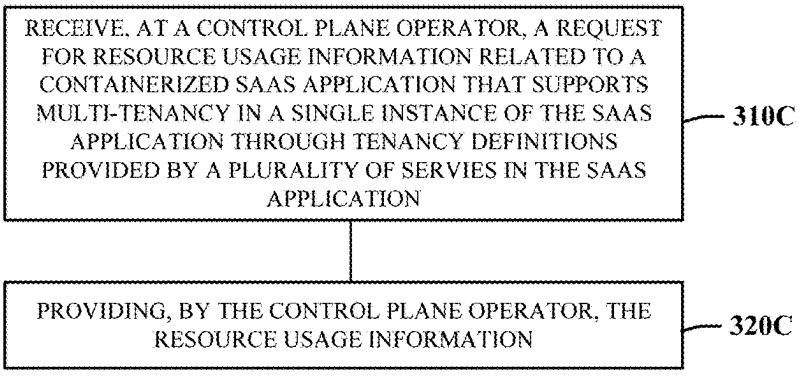
300A

FIG. 3A



300B

FIG. 3B



300C

FIG. 3C

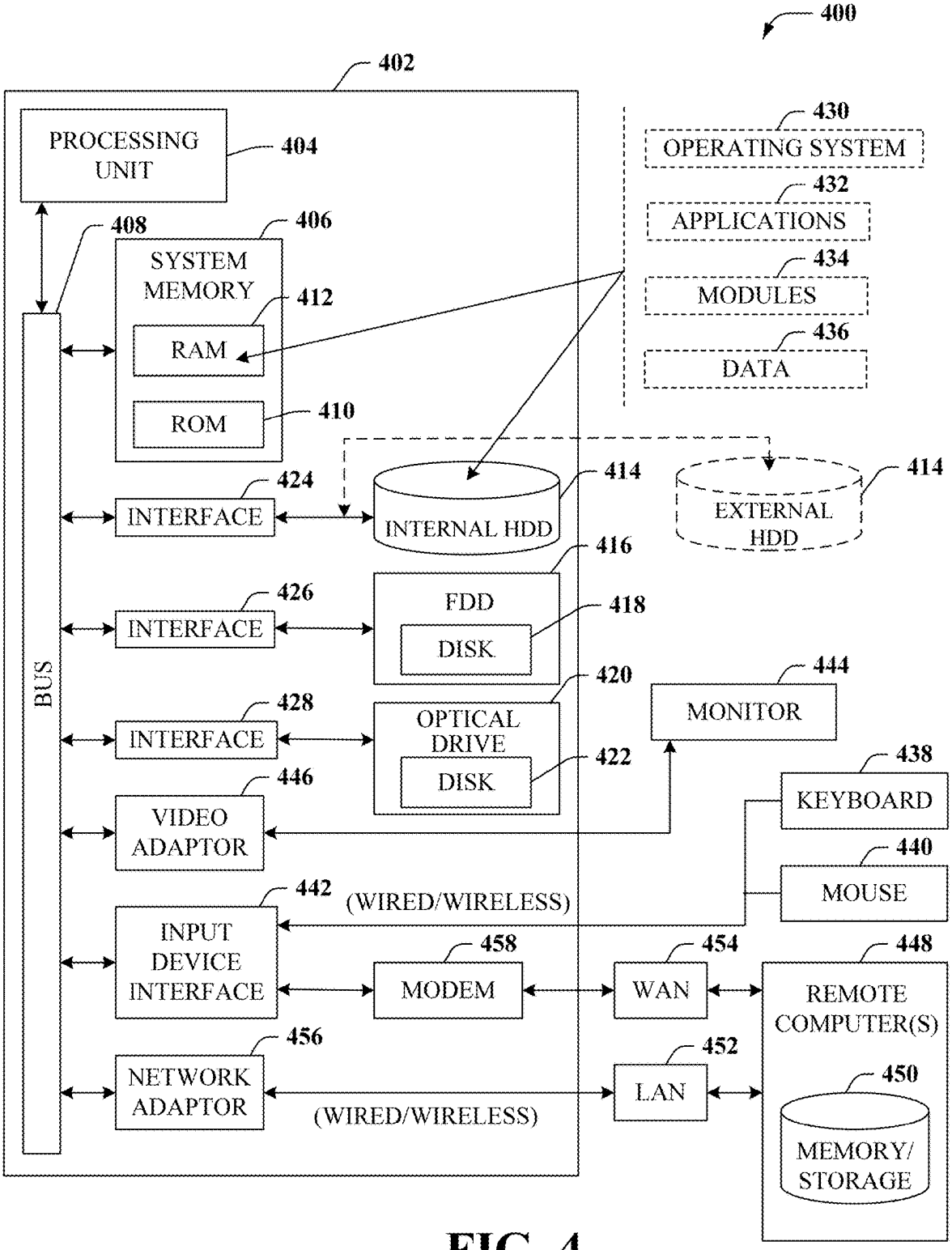

400
402
PROCESSING UNIT
404
408
406
SYSTEM MEMORY
412
RAM
ROM
410
BUS
424
INTERFACE
414
INTERNAL HDD
416
426
INTERFACE
FDD
418
DISK
420
428
INTERFACE
OPTICAL DRIVE
422
DISK
446
VIDEO ADAPTOR
442
INPUT DEVICE INTERFACE
458
MODEM
456
NETWORK ADAPTOR
(WIRED/WIRELESS)
430
OPERATING SYSTEM
432
APPLICATIONS
434
MODULES
436
DATA
414
EXTERNAL HDD
444
MONITOR
438
KEYBOARD
440
MOUSE
(WIRED/WIRELESS)
454
WAN
448
REMOTE COMPUTER(S)
452
LAN
450
MEMORY/ STORAGE

FIG. 4

TENANCY CONTROL PLANE FOR SAAS APPLICATIONS

FIELD OF THE DISCLOSURE

[0001] The subject disclosure relates to multi-tenancy Software-as-a-Service (SaaS) applications running on container orchestration platforms.

BACKGROUND

[0002] Kubernetes is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications. Kubernetes has a concept of namespaces that provides a mechanism to isolate groups of resources within a single cluster. A multi-tenant SaaS application can be implemented in Kubernetes by deploying each tenant in a different namespace to isolate resources between the tenants; however, this results in deploying an instance of the entire application in each namespace. Replicating the entire application in a different namespace for each tenant may result in wasted compute resources.

BRIEF DESCRIPTION OF THE DRAWINGS

[0003] Reference will now be made to the accompanying drawings, which are not necessarily drawn to scale, and wherein:

[0004] FIG. **1** is a block diagram illustrating an example, non-limiting embodiment of a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein.

[0005] FIGS. **2**A and **2**B are block diagrams illustrating an example, non-limiting embodiments of services operating in a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein.

[0006] FIG. **2**C is a block diagram illustrating an example, non-limiting embodiment of a sequence for tenant onboarding in a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein.

[0007] FIGS. **3**A-**3**C depict illustrative embodiments of methods in accordance with various aspects described herein.

[0008] FIG. **4** is a block diagram of an example, non-limiting embodiment of a computing environment in accordance with various aspects described herein.

DETAILED DESCRIPTION

[0009] The subject disclosure describes, among other things, illustrative embodiments for containerized SaaS applications that support multi-tenancy in a single instance of the containerized SaaS application. Other embodiments are described in the subject disclosure.

[0010] One or more aspects of the subject disclosure include a non-transitory machine-readable medium, comprising executable instructions that, when executed by a processing system including a processor, facilitate performance of operations. The operations may include receiving, at a control plane operator (e.g., **110**; FIG. **1**) running on a container orchestration platform, tenancy definitions for each of a plurality of services running in a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application; and storing the tenancy definitions for each of the plurality of services in a database accessible to the control plane operator for future creation of tenancy components for each of the plurality of services as tenants are onboarded in the containerized SaaS application.

[0011] Additional aspects of the subject disclosure include monitoring, at the control plane operator, for changes in the tenancy definitions; the receiving the tenancy definitions comprising being alerted that a custom resource definition (CRD) has been created; and the storing the tenancy definitions comprising retrieving the tenancy definitions from the CRD and storing the tenancy definitions in the database.

[0012] One or more aspects of the subject disclosure include a non-transitory machine-readable medium, comprising executable instructions that, when executed by a processing system including a processor, facilitate performance of operations. The operations may include receiving, at a control plane operator running on a container orchestration platform, an indication that a tenant is being onboarded in a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application; and creating tenancy components for each of a plurality of services in the containerized SaaS application to support the multi-tenancy in each of the plurality of services, wherein the tenancy components are defined by tenancy definitions provided by each of the plurality of services.

[0013] Additional aspects of the subject disclosure include the receiving the indication that the tenant is being onboarded comprising being alerted that a custom resource definition (CRD) for the tenant (Tenant CRD) has been created, and marking the Tenant CRD as complete in response to all tenancy components for each of the plurality of services having been created.

[0014] Additional aspects of the subject disclosure include the receiving the indication that the tenant is being onboarded comprising receiving a Kubernetes Watch event and/or polling a resource state using a Kubernetes application programming interface (API). Further additional aspects include the creating the tenancy components comprising instructing each of the plurality of services in the containerized SaaS application to create the tenancy components.

[0015] One or more aspects of the subject disclosure include a non-transitory machine-readable medium, comprising executable instructions that, when executed by a processing system including a processor, facilitate performance of operations. The operations may include receiving, at a control plane operator running on a container orchestration platform, a request for resource usage information related to a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application through tenancy definitions provided by a plurality of services in the containerized SaaS application; and providing, by the control plane operator the resource usage information.

[0016] Additional aspects of the subject disclosure include the control plane operator, the containerized SaaS application, and the plurality of services are deployed in a common Kubernetes namespace, embodiments in which the control plane operator is part of the containerized SaaS application, embodiments in which the control plane operator is part of the container orchestration platform, and embodiments in

which the control plane operator is implemented as a custom resource in a Kubernetes cluster.

[0017] Further additional aspects of the subject disclosure include methods performed as a result of the operations described above, as well as devices that perform the methods.

[0018] Various embodiments described herein provide a solution to define and manage the tenancy criteria for services managed in any container orchestration platform like Kubernetes. This disclosure describes the solutions using Kubernetes as an example; however, the various embodiments may be employed in any container orchestration platform.

[0019] Services running inside a single namespace inside a Kubernetes cluster cannot currently define the criteria based on which they want to manage different tenants. Different components or services running inside the Kubernetes cluster may want to manage different tenants in different ways. For example, in the case of Cassandra (an Apache NoSQL distributed database), an application may achieve isolation by creating different key spaces for every customer. In the case of Kafka (an Apache distributed event streaming platform), tenancy isolation may be achieved by creating different partitions for different tenants. In the case of Postgres (an open-source relational database), tenancy isolation may be achieved by providing a separate database instance for every tenant. In some embodiments, one or more services may have a requirement in which they want to have separate service instance for every tenant. The foregoing service-level multi-tenancy definitions are provided as examples. In some embodiments, each service may provide its own definitions and requirements to implement multi-tenancy.

[0020] In various embodiments, a controller (referred to herein as a "Tenant Control Plane," "Tenant Control Plane Controller," or "Tenant Control Plane Operator") is provided to manage the multiple tenants in a single instance of a SaaS application in a Kubernetes cluster in accordance with tenancy definitions provided by services. For example, the Tenant Control Plane may receive tenancy definitions provided by services, and then ensure that tenant isolation is provided when a tenant is onboarded by informing the services to create tenant resources that comply with the tenancy definitions. The Tenant Control Plane communicates with each service running inside a Kubernetes cluster and each service reports the criteria (e.g., tenancy definitions) based on how it wants to allocate resources when a new tenant is being added to the system. Based on the information it receives from the services, the Tenant Control Plane requests the Kubernetes cluster (through REST APIs) to allocate or provision the required resources inside the cluster.

[0021] Various embodiments described herein provide tenancy management at a more granular level than resources modeled by Kubernetes. For example, Kubernetes supports Role based Access Control (RBAC), but RBAC works only on the resources modelled by Kubernetes. This is in contrast to the embodiments described herein, in which multiple tenants may share resources (e.g., services) within a Kubernetes resource (e.g., the Pod).

[0022] Along with provisioning the required resources inside the Kubernetes cluster, in some embodiments, the Tenant Control Plane keeps all the information related to a particular tenant, and may provide an API support providing metrics for a specific tenant. For example, if an administrator wants to query the resources (e.g., CPU, Memory etc.) that a particular tenant is consuming, then the Tenant Control Plane layer provides a consolidated view through an API, and based on this information, the administrator can take further action if required. The Tenant Control Plane is capable of providing this view at a lower level than Kubernetes resources. This may also help the onboarding process of the Tenant. For example, the Tenant Control Plane has an upfront awareness of all the desired tenancy definitions for all the services and is capable of providing continuous updates during the tenant onboarding process. It may also help in debugging in case the onboarding process experiences issues. For example, if the onboarding process gets stuck, then the Tenant Control Plane may provide useful information that aids in identifying where/why the process is stuck. Also for example, the Tenant Control Plane operator may provide a share-of-pie analysis showing resource consumption on a tenant specific bases, and may also be used to aid in the billing process.

[0023] In some embodiments, the Tenant Control Plane may be implemented as a custom resource by using the Kubernetes API. In other embodiments, the Tenant Control Plane may be implemented as part of the containerized orchestration platform (e.g., part of the Kubernetes distribution).

[0024] As described herein, services define their tenancy definition as soon as they get deployed on the Kubernetes platform. Tenancy definition may vary for each service. This definition will be applied to each tenant as soon as it is boarded in the deployment. Later, if a service has changed its tenancy definition, then it will be redistributed from the center place only. This is a seamless workflow and services have more flexibility in tenancy models. Even introducing a new service or replacing an existing service with a new technology stack is also supported.

[0025] For example, a database service may have a tenancy definition which requires a separate database for each customer. If the SaaS application owner has decided to switch to a database which supports sharding, then the tenancy definition can be that each customer will have separate shards. This complete use case is easily handled by the embodiments described herein. Migration from an old service to a new service can also be tracked under this tenancy realization cycle.

[0026] FIG. **1** is a block diagram illustrating an example, non-limiting embodiment of a system that includes a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein. System **100** includes containerized SaaS application **150**, tenancy control plane operator **110**, database **112**, services **122**, service custom resource definitions **120**, tenant custom resource definitions **130**, tenant resources **132**, and reporting API **160**.

[0027] As shown in FIG. **1**, Tenant Control plane Operator **110** may manage multi-tenancy in a Kubernetes cluster. When an application (e.g., containerized application **150**) which supports SaaS gets deployed in any Kubernetes cluster, all of the services (also referred to herein as "microservices") **122** create their own Custom Resource Definition (Service CRD) having the information about their tenancy definition as depicted in FIG. **1** by Service CRD **120**.

[0028] Tenant Control plane Operator **110** watches for the creation of Service CRDs through the Kubernetes API server

as shown at **114**. As soon as a Service CRD gets created, Tenant Control Plane Operator **110** fetches at **123** the tenancy definition in the Service CRD and persists it in database **112**, which is accessible to Tenant Control Plane Operator **110**. In some embodiments, Tenant Control Plane Operator **110** watches for the creation of Service CRDs using Kubernetes Watch events. Also in some embodiments, Tenant Control Plane Operator **110** watches for the creation of Service CRDs by polling a resource state using a Kubernetes API.

[0029] When a tenant gets onboarded in the system, application **150** creates a new CRD instance of the tenant (Tenant CRD) **130**. Tenant Control plane Operator **110** watches for the creation of Tenant CRDs through the Kubernetes API server as shown at **116**. Once Tenant Control Plane Operator **110** receives the notification of the tenant onboarding, it triggers the creation of tenancy components in SaaS deployment as depicted at **153**. Once all the tenancy components are created in the application for a particular tenant, the Tenancy Control Plane Operator **110** marks the Tenant CRD as complete which means that the containerized SaaS application is ready to execute any workflow for that tenant.

[0030] Tenant Control Plane Operator **110** has the information of all the resources and their allocated tenants. So, in the running SaaS application if user wants to fetch information like resources consumed by a particular tenant, then it can be retrieved at API **160** as depicted in FIG. **1** at **163**.

[0031] FIGS. **2**A and **2**B are block diagrams illustrating an example, non-limiting embodiments of services operating in a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein. The services shown in FIGS. **2**A and **2**B, and their manner of implementing multi-tenancy are examples. Services are free to implement multi-tenancy in any manner through the creation of Service CRDs.

[0032] FIG. **2**A shows an example implementation of a multi-tenancy Kafka service. In this example, the Kafka service, when deployed, creates a Service CRD with a tenancy definition that requires a separate Kafka topic to be created for each tenant (or "customer") such that a tenant identifier is embedded in the topic name, and defined topics have a replication factor of two. This is shown in FIG. **2**A with customers **232**A, **234**A, and **236**A, each having access to a common Kafka service in the SaaS application, but with isolation provided by the resources created according to tenancy definitions in the Service CRD created by the Kafka service. Three Kafka pods **212**A, **214**A, **216**A, are created, in a manner that supports the replication factor of two.

[0033] Other example tenancy definitions may include a topic being limited to using not more than 20% of the available capacity of Kafka (governed by the Kafka quota provided by the service, not by the container orchestration platform). Additional tenancy definitions may include rules governing lag not being increased by a certain threshold value of X, or rules governing scaling or tenant redistribution.

[0034] In some embodiments, if backpressure reaches a certain defined limit of X or the size of the partitions is increased beyond a defined limit, then traffic for that customer can either be redistributed to other Kafka clusters or new Kafka clusters can be spawned. Also in some embodiments, if backpressure goes down, then Kafka clusters can be scaled down as well.

[0035] The Tenancy Control Plane view in FIG. **2**A is showing different customers like TC 1 Customer, TC 2 Customer and TC 3 Customer, and the Kubernetes view in FIG. **2**A is showing a single Kafka service cluster (cluster of 3 nodes) which is serving all three customers. Accordingly, Kubernetes sees a single application with a single Kafka service, whereas the Tenancy Control Plane Operator sees a multi-tenancy SaaS application serving multiple customers (tenants) with services that define their own multi-tenancy schemes through custom resource definitions.

[0036] FIG. **2**B shows an example implementation of a multi-tenancy Resource Adapter Service. The role of this service is to listen to the data on the socket for various protocols (Like TL1, Netconf, gnmi etc.). It reads data from different customer devices and puts them in a dedicated buffer defined per customer. As per this design, there is no need to have multiple services running to hear the data from the network until there is heavy traffic coming from the network. So, traffic level permitting, the maximum number of customers may be managed by a single service instance. According to the Service CRD for the resource adapter service, the events are segregated and isolation between different customer devices is provided. In the example of FIG. **2**B, the isolation of a single tenant's resources is shown. Event buffer **220** is a buffer dedicated to a single tenant that desires to listen for traffic from devices **212**B, **214**B, and **216**B. The traffic from these devices is placed in the event buffer **220** which is dedicated to the same tenant.

[0037] In this example, the resource adapter service, when deployed, creates a Service CRD with a tenancy definition that requires a dedicated buffer to be created per customer. Thes size or bouncing limit of the buffer can be defined as per the customer profile. For example, a buffer size for a "Gold" level customer may be higher than for a "Bronze" level customer. Other example tenancy definitions may include the buffer size being kept to a particular maximum threshold size. If it goes beyond that then the appropriate algorithm may be used to drop the events. Additional tenancy definitions may include rules governing scaling or redistribution. For example, if traffic is high from the network then buffers can report that information and another instance of the service may be spawned to take care of load. The same in case a downscale is required.

[0038] Any type of service may provide tenant definitions and support multi-tenancy. For example, an Orchestrator Service may support multi-tenancy. In this example, the service takes care of all the provisioning requests coming from either the REST interface or UI. These requests may belong to multiple customers. In some embodiments, this service may use a concept of the domain per customer. In these embodiments, the service may create a logical group referred to as "domains" that provides an isolation of data for different customers. For example, domain X data should be completely isolated and should not impact domain Y in any way whether it is in terms of consuming resources or computation. As an example, in NMS systems, practically there is not much provisioning or high requests so, in spite of having separate services for each customer, single service with multiple domains can fulfill the requirement which is cost-effective and easy to manage.

[0039] In this example, the orchestrator service, when deployed, creates a Service CRD with a tenancy definition that requires separate domains to be created for each cus-

tomer, where requests persisted or processed within a domain are fixed or configurable.

[0040] Other example tenancy definitions may include a number of requests that can be processed in each domain at any given point in time can be made configurable. Additional tenancy definitions may include rules governing scaling or redistribution. For example, if the number of requests from northbound increases and the configured rate is not able to support, then another instance of the orchestrator service may be launched.

[0041] In another example, a Postgres Service may support multi-tenancy. In this example, each customer may have a separate database with the same schema, where each database name includes a tenant identifier to provide isolation. Additional tenancy definitions may include rules governing scaling or redistribution. For example, the size of the per-customer database may be limited to not increase by factor X, and/or a read query per customer database may be limited to Y rate/sec in case all tenant is providing the same load, and/or the Upsert rate per customer database may be limited to Z. Any of the above may be cross-checked with the pg_stat activity table, but this is one of the ways that services are free to implement their own multi-tenancy definitions. Additional tenancy definitions may include rules governing scaling or redistribution. For example, if the customer database is not able to meet the read requirements, then it may be time to HA scale the replica node. Also for example, if the size of the database increases with the decided limit then this customer database may be shipped to a different cluster.

[0042] FIG. **2**C is a block diagram illustrating an example, non-limiting embodiment of a sequence for tenant onboarding in a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application in accordance with various aspects described herein. As shown in FIG. **2**C, onboarding operator **210**C represents the tenant control plane operator and operations taken thereby with respect to onboarding a tenant. In the context of FIG. **2**C, a tenant is onboarded in a SaaS application having four services: Kafka service **220**C, Postgres service **230**C, RA service **240**C, and orchestrator service **250**C. The number of services in FIG. **2**C is purposely kept to a small number for illustration purposes. In some embodiments, the number of services that exist within a containerized multi-tenant SaaS application may be in the hundreds or even thousands.

[0043] The operations shown in FIG. **2**C take place after a SaaS application has been deployed, and all of the services within the SaaS application have created their own Service CRD with tenant definitions. The tenant control plane operator has (e.g., through a watch event, or by polling) been notified of the creation of all of the Service CRDs associated with the services, and persisted that information in its own accessible database. The SaaS application has started the process of onboarding a new tenant, and has created a Tenant CRD. The tenant control plane operator has (e.g., through a watch event, or by polling) been notified that a new tenant is being onboarded. It is at this point that the operations in FIG. **2**C take place to onboard the tenant in each of the services according to their own tenant definitions.

[0044] In response to an indication that a new tenant is being onboarded, the onboarding operator **210**C within the tenant control plane operator, alerts the Kafka service **220**C to perform operations to create tenant resources within the service to provide isolation for the tenant. For example, the onboarding operator **210**C may provide a tenant identifier, such as a tenant name, to Kafka service **220**C, and Kafka service **220**C may create a new tenant topic and tune the Kafka topic quota at **222**C using the tenant identifier. Once the Kafka service has completed the tenant onboarding it returns an indication to onboarding operator **210**C that it is complete.

[0045] Similar operations are performed with all remaining services that have created Service CRDs with tenant definitions. For example, onboarding operator **210**C alerts Postgres service **230**C, and Postgres service **230**C creates a new database and configures quotas and other parameters at **232**C. Similarly, onboarding operator **210**C alerts RA service **240**C, which creates a buffer for the tenant at **242**C. Also similarly, onboarding operator **210**C alerts orchestrator service **250**C, which creates a domain at **252**C.

[0046] When onboarding operator **210**C has completed the tenant onboarding for each of the services that created service CRDs with tenant definitions, the tenant control plane operator marks the tenant CRD as complete, thereby notifying the SaaS application that the onboarding process is complete and that workflow may begin.

[0047] Also shown in FIG. **2**C is a particular customer reporting a drop in read rate in the Postgres service **230**C at **234**C. In response, onboarding operator **210**C may make a call to Kubernetes API **260**C to scale up a Postgres node at **262**C.

[0048] FIGS. **3**A-**3**C depict illustrative embodiments of methods in accordance with various aspects described herein. Method **300**A in FIG. **3**A represents actions taken when a multi-tenancy containerized SaaS application is deployed, and the SaaS application has multiple services that can define their own multi-tenancy requirements through tenant definitions in Service CRDs.

[0049] At **310**A, a control plane operator receives tenancy definitions for each of a plurality of services running in a containerized SAS application that supports multi-tenancy in a single instance of the SaaS application. Referring back to FIG. **1**, in some embodiments, this corresponds to tenancy control plane operator **110** receiving tenancy definitions in Service CRDs created by services within SaaS application **150** when they are deployed. In some embodiments, the tenant control plane operator may be alerted that a Service CRD has been created. For example, the tenant control plane operator may set up a watch event, such that the tenant control plane operator is alerted whenever a Service CRD is created. Once the tenant control plane operator is alerted that a service CRD has been created, the control plane operator may retrieve the contents of the service CRD using the Kubernetes API.

[0050] At **320**A, the tenancy definitions for each of the services are stored in a database accessible to the control plane operator for future creation of tenancy components for each of the services as tenants are onboarded in the SAS application. Referring back to FIG. **1**, in some embodiments, this corresponds to storing the tenancy definitions in database **112**, which is accessible to tenancy control plane operator **110**. In some embodiments, the tenancy definitions stored in database **112** are used in the onboarding process when communicating with each of the services. Examples of these communications are described above with reference to FIG. **2**C.

[0051] Method **300**B in FIG. **3**B represents actions taken when a tenant is onboarded in a containerized SaaS appli-

cation that supports multi-tenancy in a single instance of the containerized SaaS application. At **310**B, a control plane operator receives an indication that a tenant is being onboarded in a containerized SaaS application that supports multi-tenancy in a single instance of the containerized SaaS application. Referring back to FIG. **1**, this may correspond to a tenant control plane operator **110** receiving an indication that SaaS application **150** has created a Tenant CRD **130** for a new tenant that is being onboarded. In some embodiments, tenant control plane operator **110** may be alerted of the new Tenant CRD through the use of a watch event **116**.

[0052] At **320**B, tenancy components are created for each of a plurality of services in the containerized SaaS application to support the multi-tenancy in each of the services, wherein the tenancy components are defined by tenant definitions provided by each of the services. In some embodiments, the actions of **320**B correspond to the actions shown in FIG. **2**C, in which an onboarding operator within the tenancy control plane operator alerts each of the services that has created a Service CRD with a tenant definition that a new customer is being onboarded.

[0053] Method **300**C in FIG. **3**C represents actions taken when a request for resource usage is received at a control plane operator. At **310**C, a control plane operator receives a request for resource usage information related to a containerized SaaS application that supports multi-tenancy in a single instance of the SaaS application through tenancy definitions provided by a plurality of services in the SaaS application. Referring now back to FIG. **1**, this corresponds to receiving, at API **160**, a request for resource usage by a tenant. At **320**C, the control plane operator provides the resource usage information requested at **310**C.

[0054] While for purposes of simplicity of explanation, the respective processes are shown and described as a series of blocks in FIGS. **3**A-**3**C, it is to be understood and appreciated that the claimed subject matter is not limited by the order of the blocks, as some blocks may occur in different orders and/or concurrently with other blocks from what is depicted and described herein. Moreover, not all illustrated blocks may be required to implement the methods described herein.

[0055] Turning now to FIG. **4**, there is illustrated a block diagram of a computing environment in accordance with various aspects described herein. In order to provide additional context for various embodiments of the embodiments described herein, FIG. **4** and the following discussion are intended to provide a brief, general description of a suitable computing environment **400** in which the various embodiments of the subject disclosure can be implemented. For example, computing environment **400** can facilitate in whole or in part the deployment of a containerized multi-tenancy SaaS application in a container orchestration platform. Multiple services may be included in the SaaS application, and the multiple services, upon deployment, may create Service CRDs with tenant definitions. A control plane operator may be alerted that service CRDs have been created, and may fetch and store them in persistent storage accessible to the tenant control plane operator. The SaaS application may onboard a tenant, and create a Tenant CRD for the new tenant. The tenant control plane operator may be alerted of the new tenant being onboarded, and may alert each of the services that created a Service CRD with a tenant definition. The tenant control plane operator may mark the Tenant CRD as complete, and workflow for the onboarded tenant may begin.

[0056] Generally, program modules comprise routines, programs, components, data structures, etc., that perform particular tasks or implement particular abstract data types. Moreover, those skilled in the art will appreciate that the methods can be practiced with other computer system configurations, comprising single-processor or multiprocessor computer systems, minicomputers, mainframe computers, as well as personal computers, hand-held computing devices, microprocessor-based or programmable consumer electronics, and the like, each of which can be operatively coupled to one or more associated devices.

[0057] As used herein, a processing circuit includes one or more processors as well as other application specific circuits such as an application specific integrated circuit, digital logic circuit, state machine, programmable gate array or other circuit that processes input signals or data and that produces output signals or data in response thereto. It should be noted that while any functions and features described herein in association with the operation of a processor could likewise be performed by a processing circuit.

[0058] The illustrated embodiments of the embodiments herein can be also practiced in distributed computing environments where certain tasks are performed by remote processing devices that are linked through a communications network. In a distributed computing environment, program modules can be located in both local and remote memory storage devices.

[0059] Computing devices typically comprise a variety of media, which can comprise computer-readable storage media and/or communications media, which two terms are used herein differently from one another as follows. Computer-readable storage media can be any available storage media that can be accessed by the computer and comprises both volatile and nonvolatile media, removable and non-removable media. By way of example, and not limitation, computer-readable storage media can be implemented in connection with any method or technology for storage of information such as computer-readable instructions, program modules, structured data or unstructured data.

[0060] Computer-readable storage media can comprise, but are not limited to, random access memory (RAM), read only memory (ROM), electrically erasable programmable read only memory (EEPROM), flash memory or other memory technology, compact disk read only memory (CD ROM), digital versatile disk (DVD) or other optical disk storage, magnetic cassettes, magnetic tape, magnetic disk storage or other magnetic storage devices or other tangible and/or non-transitory media which can be used to store desired information. In this regard, the terms "tangible" or "non-transitory" herein as applied to storage, memory or computer-readable media, are to be understood to exclude only propagating transitory signals per se as modifiers and do not relinquish rights to all standard storage, memory or computer-readable media that are not only propagating transitory signals per se.

[0061] Computer-readable storage media can be accessed by one or more local or remote computing devices, e.g., via access requests, queries or other data retrieval protocols, for a variety of operations with respect to the information stored by the medium.

[0062] Communications media typically embody computer-readable instructions, data structures, program modules or other structured or unstructured data in a data signal such as a modulated data signal, e.g., a carrier wave or other transport mechanism, and comprises any information delivery or transport media. The term "modulated data signal" or signals refers to a signal that has one or more of its characteristics set or changed in such a manner as to encode information in one or more signals. By way of example, and not limitation, communication media comprise wired media, such as a wired network or direct-wired connection, and wireless media such as acoustic, RF, infrared and other wireless media.

[0063] With reference again to FIG. **4**, the example environment can comprise a computer **402**, the computer **402** comprising a processing unit **404**, a system memory **406** and a system bus **408**. The system bus **408** couples system components including, but not limited to, the system memory **406** to the processing unit **404**. The processing unit **404** can be any of various commercially available processors. Dual microprocessors and other multiprocessor architectures can also be employed as the processing unit **404**.

[0064] The system bus **408** can be any of several types of bus structure that can further interconnect to a memory bus (with or without a memory controller), a peripheral bus, and a local bus using any of a variety of commercially available bus architectures. The system memory **406** comprises ROM **410** and RAM **412**. A basic input/output system (BIOS) can be stored in a non-volatile memory such as ROM, erasable programmable read only memory (EPROM), EEPROM, which BIOS contains the basic routines that help to transfer information between elements within the computer **402**, such as during startup. The RAM **412** can also comprise a high-speed RAM such as static RAM for caching data.

[0065] The computer **402** further comprises an internal hard disk drive (HDD) **414** (e.g., EIDE, SATA), which internal HDD **414** can also be configured for external use in a suitable chassis (not shown), a magnetic floppy disk drive (FDD) **416**, (e.g., to read from or write to a removable diskette **418**) and an optical disk drive **420**, (e.g., reading a CD-ROM disk **422** or, to read from or write to other high-capacity optical media such as the DVD). The HDD **414**, magnetic FDD **416** and optical disk drive **420** can be connected to the system bus **408** by a hard disk drive interface **424**, a magnetic disk drive interface **426** and an optical drive interface **428**, respectively. The hard disk drive interface **424** for external drive implementations comprises at least one or both of Universal Serial Bus (USB) and Institute of Electrical and Electronics Engineers (IEEE) 1394 interface technologies. Other external drive connection technologies are within contemplation of the embodiments described herein.

[0066] The drives and their associated computer-readable storage media provide nonvolatile storage of data, data structures, computer-executable instructions, and so forth. For the computer **402**, the drives and storage media accommodate the storage of any data in a suitable digital format. Although the description of computer-readable storage media above refers to a hard disk drive (HDD), a removable magnetic diskette, and a removable optical media such as a CD or DVD, it should be appreciated by those skilled in the art that other types of storage media which are readable by a computer, such as zip drives, magnetic cassettes, flash memory cards, cartridges, and the like, can also be used in the example operating environment, and further, that any such storage media can contain computer-executable instructions for performing the methods described herein.

[0067] A number of program modules can be stored in the drives and RAM **412**, comprising an operating system **430**, one or more application programs **432**, other program modules **434** and program data **436**. All or portions of the operating system, applications, modules, and/or data can also be cached in the RAM **412**. The systems and methods described herein can be implemented utilizing various commercially available operating systems or combinations of operating systems.

[0068] A user can enter commands and information into the computer **402** through one or more wired/wireless input devices, e.g., a keyboard **438** and a pointing device, such as a mouse **440**. Other input devices (not shown) can comprise a microphone, an infrared (IR) remote control, a joystick, a game pad, a stylus pen, touch screen or the like. These and other input devices are often connected to the processing unit **404** through an input device interface **442** that can be coupled to the system bus **408**, but can be connected by other interfaces, such as a parallel port, an IEEE 1394 serial port, a game port, a universal serial bus (USB) port, an IR interface, etc.

[0069] A monitor **444** or other type of display device can be also connected to the system bus **408** via an interface, such as a video adapter **446**. It will also be appreciated that in alternative embodiments, a monitor **444** can also be any display device (e.g., another computer having a display, a smart phone, a tablet computer, etc.) for receiving display information associated with computer **402** via any communication means, including via the Internet and cloud-based networks. In addition to the monitor **444**, a computer typically comprises other peripheral output devices (not shown), such as speakers, printers, etc.

[0070] The computer **402** can operate in a networked environment using logical connections via wired and/or wireless communications to one or more remote computers, such as a remote computer(s) **448**. The remote computer(s) **448** can be a workstation, a server computer, a router, a personal computer, portable computer, microprocessor-based entertainment appliance, a peer device or other common network node, and typically comprises many or all of the elements described relative to the computer **402**, although, for purposes of brevity, only a remote memory/storage device **450** is illustrated. The logical connections depicted comprise wired/wireless connectivity to a local area network (LAN) **452** and/or larger networks, e.g., a wide area network (WAN) **454**. Such LAN and WAN networking environments are commonplace in offices and companies, and facilitate enterprise-wide computer networks, such as intranets, all of which can connect to a global communications network, e.g., the Internet.

[0071] When used in a LAN networking environment, the computer **402** can be connected to the LAN **452** through a wired and/or wireless communication network interface or adapter **456**. The adapter **456** can facilitate wired or wireless communication to the LAN **452**, which can also comprise a wireless AP disposed thereon for communicating with the adapter **456**.

[0072] When used in a WAN networking environment, the computer **402** can comprise a modem **458** or can be connected to a communications server on the WAN **454** or has other means for establishing communications over the WAN

454, such as by way of the Internet. The modem **458**, which can be internal or external and a wired or wireless device, can be connected to the system bus **408** via the input device interface **442**. In a networked environment, program modules depicted relative to the computer **402** or portions thereof, can be stored in the remote memory/storage device **450**. It will be appreciated that the network connections shown are example and other means of establishing a communications link between the computers can be used.

[0073] The computer **402** can be operable to communicate with any wireless devices or entities operatively disposed in wireless communication, e.g., a printer, scanner, desktop and/or portable computer, portable data assistant, communications satellite, any piece of equipment or location associated with a wirelessly detectable tag (e.g., a kiosk, news stand, restroom), and telephone. This can comprise Wireless Fidelity (Wi-Fi) and BLUETOOTH® wireless technologies. Thus, the communication can be a predefined structure as with a conventional network or simply an ad hoc communication between at least two devices.

[0074] Wi-Fi can allow connection to the Internet from a couch at home, a bed in a hotel room or a conference room at work, without wires. Wi-Fi is a wireless technology similar to that used in a cell phone that enables such devices, e.g., computers, to send and receive data indoors and out; anywhere within the range of a base station. Wi-Fi networks use radio technologies called IEEE 802.11 (a, b, g, n, ac, ag, etc.) to provide secure, reliable, fast wireless connectivity. A Wi-Fi network can be used to connect computers to each other, to the Internet, and to wired networks (which can use IEEE 802.3 or Ethernet). Wi-Fi networks operate in the unlicensed 2.4 and 5 GHz radio bands for example or with products that contain both bands (dual band), so the networks can provide real-world performance similar to the basic 10BaseT wired Ethernet networks used in many offices.

[0075] What has been described above includes mere examples of various embodiments. It is, of course, not possible to describe every conceivable combination of components or methodologies for purposes of describing these examples, but one of ordinary skill in the art can recognize that many further combinations and permutations of the present embodiments are possible. Accordingly, the embodiments disclosed and/or claimed herein are intended to embrace all such alterations, modifications and variations that fall within the spirit and scope of the appended claims. Furthermore, to the extent that the term "includes" is used in either the detailed description or the claims, such term is intended to be inclusive in a manner similar to the term "comprising" as "comprising" is interpreted when employed as a transitional word in a claim.

[0076] Computing devices typically comprise a variety of media, which can comprise computer-readable storage media and/or communications media, which two terms are used herein differently from one another as follows. Computer-readable storage media can be any available storage media that can be accessed by the computer and comprises both volatile and nonvolatile media, removable and non-removable media. By way of example, and not limitation, computer-readable storage media can be implemented in connection with any method or technology for storage of information such as computer-readable instructions, program modules, structured data or unstructured data. Computer-readable storage media can comprise the widest variety of storage media including tangible and/or non-transitory media which can be used to store desired information. In this regard, the terms "tangible" or "non-transitory" herein as applied to storage, memory or computer-readable media, are to be understood to exclude only propagating transitory signals per se as modifiers and do not relinquish rights to all standard storage, memory or computer-readable media that are not only propagating transitory signals per se.

[0077] In addition, a flow diagram may include a "start" and/or "continue" indication. The "start" and "continue" indications reflect that the steps presented can optionally be incorporated in or otherwise used in conjunction with other routines. In this context, "start" indicates the beginning of the first step presented and may be preceded by other activities not specifically shown. Further, the "continue" indication reflects that the steps presented may be performed multiple times and/or may be succeeded by other activities not specifically shown. Further, while a flow diagram indicates a particular ordering of steps, other orderings are likewise possible provided that the principles of causality are maintained.

[0078] As may also be used herein, the term(s) "operably coupled to", "coupled to", and/or "coupling" includes direct coupling between items and/or indirect coupling between items via one or more intervening items. Such items and intervening items include, but are not limited to, junctions, communication paths, components, circuit elements, circuits, functional blocks, and/or devices. As an example of indirect coupling, a signal conveyed from a first item to a second item may be modified by one or more intervening items by modifying the form, nature or format of information in a signal, while one or more elements of the information in the signal are nevertheless conveyed in a manner than can be recognized by the second item. In a further example of indirect coupling, an action in a first item can cause a reaction on the second item, as a result of actions and/or reactions in one or more intervening items.

[0079] Although specific embodiments have been illustrated and described herein, it should be appreciated that any arrangement which achieves the same or similar purpose may be substituted for the embodiments described or shown by the subject disclosure. The subject disclosure is intended to cover any and all adaptations or variations of various embodiments. Combinations of the above embodiments, and other embodiments not specifically described herein, can be used in the subject disclosure. For instance, one or more features from one or more embodiments can be combined with one or more features of one or more other embodiments. In one or more embodiments, features that are positively recited can also be negatively recited and excluded from the embodiment with or without replacement by another structural and/or functional feature. The steps or functions described with respect to the embodiments of the subject disclosure can be performed in any order. The steps or functions described with respect to the embodiments of the subject disclosure can be performed alone or in combination with other steps or functions of the subject disclosure, as well as from other embodiments or from other steps that have not been described in the subject disclosure. Further, more than or less than all of the features described with respect to an embodiment can also be utilized.

What is claimed is:

1. A non-transitory machine-readable medium, comprising executable instructions that, when executed by a pro-

cessing system including a processor, facilitate performance of operations, the operations comprising:

receiving, at a control plane operator running on a container orchestration platform, an indication that a tenant is being onboarded in a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application; and

creating tenancy components for each of a plurality of services in the containerized SaaS application to support the multi-tenancy in each of the plurality of services, wherein the tenancy components are defined by tenancy definitions provided by each of the plurality of services.

2. The non-transitory machine-readable medium of claim **1**, wherein the receiving the indication that the tenant is being onboarded comprises being alerted that a custom resource definition (CRD) for the tenant (Tenant CRD) has been created.

3. The non-transitory machine-readable medium of claim **2**, wherein the operations further comprise marking the Tenant CRD as complete in response to all tenancy components for each of the plurality of services having been created.

4. The non-transitory machine-readable medium of claim **1**, wherein the receiving the indication that the tenant is being onboarded comprises polling a resource state through a Kubernetes application programming interface (API).

5. The non-transitory machine-readable medium of claim **1**, wherein the receiving the indication that the tenant is being onboarded comprises receiving a Kubernetes Watch event.

6. The non-transitory machine-readable medium of claim **1**, wherein the creating the tenancy components comprises instructing each of the plurality of services in the containerized SaaS application to create the tenancy components.

7. The non-transitory machine-readable medium of claim **1**, wherein the control plane operator is part of the containerized SaaS application.

8. The non-transitory machine-readable medium of claim **1**, wherein the control plane operator is part of the container orchestration platform.

9. The non-transitory machine-readable medium of claim **1**, wherein the control plane operator is implemented as a custom resource operator in a Kubernetes cluster.

10. The non-transitory machine-readable medium of claim **1**, wherein the control plane operator, the containerized SaaS application, and the plurality of services are deployed in a common Kubernetes namespace.

11. A non-transitory machine-readable medium, comprising executable instructions that, when executed by a processing system including a processor, facilitate performance of operations, the operations comprising:

receiving, at a control plane operator running on a container orchestration platform, tenancy definitions for each of a plurality of services running in a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application; and

storing the tenancy definitions for each of the plurality of services in a database accessible to the control plane operator for future creation of tenancy components for each of the plurality of services as tenants are onboarded in the containerized SaaS application.

12. The non-transitory machine-readable medium of claim **11**, wherein the operations further comprise monitoring, at the control plane operator, for changes in the tenancy definitions.

13. The non-transitory machine-readable medium of claim **11**, wherein the receiving the tenancy definitions comprises being alerted that a custom resource definition (CRD) has been created.

14. The non-transitory machine-readable medium of claim **13**, wherein the storing the tenancy definitions comprises retrieving the tenancy definitions from the CRD and storing the tenancy definitions in the database.

15. The non-transitory machine-readable medium of claim **11**, wherein the control plane operator is part of the containerized SaaS application.

16. The non-transitory machine-readable medium of claim **11**, wherein the control plane operator is part of the container orchestration platform.

17. The non-transitory machine-readable medium of claim **11**, wherein the control plane operator is implemented as a custom resource operator in a Kubernetes cluster.

18. The non-transitory machine-readable medium of claim **11**, wherein the control plane operator, the containerized SaaS application, and the plurality of services are deployed in a common Kubernetes namespace.

19. A non-transitory machine-readable medium, comprising executable instructions that, when executed by a processing system including a processor, facilitate performance of operations, the operations comprising:

receiving, at a control plane operator running on a container orchestration platform, a request for resource usage information related to a containerized Software-as-a-Service (SaaS) application that supports multi-tenancy in a single instance of the containerized SaaS application through tenancy definitions provided by a plurality of services in the containerized SaaS application; and

providing, by the control plane operator, the resource usage information.

20. The non-transitory machine-readable medium of claim **19**, wherein the control plane operator, the containerized SaaS application, and the plurality of services are deployed in a common Kubernetes namespace.

* * * * *